

Deklaration von künstlicher Intelligenz

Promptprotokoll

Promptprotokoll

Das Promptprotokoll beinhaltet die exakten Fragen an die KI sowie die vollständigen Antworten. Gemäss den Vorgaben muss dieses Protokoll lediglich digital bei der betreuenden Lehrperson eingereicht werden.

P01 – Gemini 2.0 Flash (02.12.2025)

Beschreibung: Herleitung der quadratischen Phasenfunktion des Chirps, Umformung in ein komplexes Signal sowie Erklärung der Up- und Down-Chirp-Darstellung. Die Herleitung wurde anschliessend selbst überprüft.

Exakter Prompt:

Wie komme ich von der abstrakten RF-Formel für LoRa zum eigentlichen komplexen Basisbandsignal? Und wie wende ich darauf die Eulersche Formel an, um I und Q sauber aufzutrennen?

Antwort der KI:

Um das Basisbandsignal aus der allgemeinen LoRa-RF-Formel herzuleiten und in die I/Q-Darstellung zu überführen, müssen wir schrittweise vom hochfrequenten Passbandsignal zum komplexen Basisbandsignal (I/Q) gehen.

1. Das RF-Passbandsignal Das LoRa-Signal basiert auf der *Chirp Spread Spectrum* (CSS) Modulation. Ein Signal im Passband lässt sich allgemein darstellen als:

$$s_{RF}(t) = \Re \left\{ s_b(t) \cdot e^{j2\pi f_c t} \right\}$$

Hierbei ist f_c die Trägerfrequenz (z. B. 868 MHz oder 2.4 GHz) und $s_b(t)$ das gesuchte komplexe Basisbandsignal.

2. Frequenz- und Phasenverlauf eines Chirps Ein einzelner LoRa Up-Chirp hat eine linear ansteigende Momentanfrequenz. Innerhalb eines Symbolintervalls T_s überstreicht die Frequenz die gesamte Bandbreite B . Die Momentanfrequenz $f(t)$ lässt sich durch eine Geradengleichung beschreiben:

$$f(t) = k \cdot t + f_{start}$$

Wobei k die Steigung und f_{start} die Startfrequenz darstellt. Da die Frequenz die zeitliche Ableitung der Phase ist ($f(t) = \frac{1}{2\pi} \frac{d\Phi}{dt}$), erhalten wir die Phase $\Phi(t)$ durch Integration der Frequenz. Geometrisch entspricht die Phase der Fläche unter der Frequenzkurve:

$$\Phi(t) = 2\pi \int_0^t f(\tau) d\tau = 2\pi \int_0^t (k \cdot \tau + f_{start}) d\tau = 2\pi \left(\frac{1}{2} k t^2 + f_{start} t \right)$$

3. Das komplexe Basisbandsignal Das Basisbandsignal $s(t)$ (bzw. $s_b(t)$) ist nun einfach der Phasenzeiger (Phasor) mit dieser berechneten Phase:

$$s(t) = e^{j\Phi(t)} = e^{j2\pi \left(\frac{1}{2} k t^2 + f_{start} t \right)}$$

4. I/Q-Darstellung mit der Eulerschen Formel Um dieses Signal nun im Computer (z. B. per Software Defined Radio) verarbeiten zu können, müssen wir es in reelle Zahlenreihen aufteilen. Hier kommt die Eulersche Formel ins Spiel: $e^{jx} = \cos(x) + j \sin(x)$. Angewendet auf unser Basisbandsignal erhalten wir die In-Phase-Komponente (I) und die Quadratur-Komponente (Q):

$$I(t) = \Re\{s(t)\} = \cos(\pi k t^2 + 2\pi f_{start} t)$$
$$Q(t) = \Im\{s(t)\} = \sin(\pi k t^2 + 2\pi f_{start} t)$$

Diese beiden Signale $I(t)$ und $Q(t)$ sind genau die Werte, die du vom ADC deines SDRs als komplexe Samples erhältst.

Up-Chirp vs. Down-Chirp Ein Down-Chirp unterscheidet sich mathematisch nur durch eine fallende Frequenzrampe. Die Steigung k wird negativ. Dadurch dreht sich das Vorzeichen vor dem quadratischen Term, was bedeutet, dass sich der Phasenzeiger in die entgegengesetzte Richtung beschleunigt. Dies erreichst du elegant durch komplexe Konjugation des Basis-Up-Chirps.

P02 – Gemini 2.0 Flash (02.12.2025)

Beschreibung: Erklärung von In-Phase- und Quadratur-Komponente sowie ihrer Darstellung als komplexe Zahl. Die Grundlagen wurden für die eigene technische Erklärung verwendet.

Exakter Prompt:

Was genau bedeuten diese I- und Q-Signale in der digitalen Signalverarbeitung?
Kannst du mir das Konzept von komplexen Samples anschaulich erklären, vielleicht mit einer geometrischen Vorstellung?

Antwort der KI:

I- und Q-Signale sind das absolute Fundament der modernen digitalen Signalverarbeitung und Funktechnik (SDR).

Was bedeuten I und Q?

- **I steht für In-Phase:** Dies ist die reale Komponente eines Signals. Man kann sie sich als eine Kosinus-Welle vorstellen.
- **Q steht für Quadratur:** Dies ist die imaginäre Komponente. Sie entspricht einer Sinus-Welle, die exakt um 90° (also eine Viertelperiode oder "in Quadratur") zur I-Komponente phasenverschoben ist.

Anschauliches Verständnis: Der rotierende Vektor Der beste Weg, sich I/Q-Samples vorzustellen, ist ein Vektor in einem zweidimensionalen Koordinatensystem (dem *I/Q-Diagramm* oder Konstellationsdiagramm).

- Die X-Achse ist die I-Achse (Realfeld).
- Die Y-Achse ist die Q-Achse (Imaginärfeld).

Jedes I/Q-Sample ($I + jQ$) repräsentiert einen Punkt in diesem Koordinatensystem. Zieht man einen Pfeil vom Ursprung (0,0) zu diesem Punkt, repräsentiert dieser Pfeil den aktuellen Zustand deiner elektromagnetischen Welle: 1. **Amplitude (Stärke der Welle)**: Entspricht der Länge des Pfeils.

$$A = \sqrt{I^2 + Q^2}$$

2. **Phase (Aktueller Schwingungszustand)**: Entspricht dem Winkel des Pfeils relativ zur positiven X-Achse.

$$\phi = \arctan 2(Q, I)$$

Warum dieser "Trick" mit komplexen Zahlen? Stell dir vor, du willst ein 868 MHz Signal (wie LoRa) aufzeichnen. Nach dem Nyquist-Shannon-Abtasttheorem müsstest du mit über 1.7 GHz abtasten – viel zu schnell für normale Computer. Das SDR mischt das Signal daher analog herunter ins Basisband (Zentrumsfrequenz wird zu 0 Hz). Würdest du beim Heruntermischen nur eine normale, reelle Abtastung (nur I) machen, könntest du positive und negative Frequenzen im Spektrum nicht mehr voneinander unterscheiden (sie falten sich übereinander). Indem das SDR das empfangene Signal einmal mit einem lokalen Oszillator (Kosinus) mischt (ergibt I) und einmal mit einem um 90° verschobenen Oszillator (Sinus) mischt (ergibt Q), erhält man zwei Signalströme. Zusammen als komplexe Zahl $I + jQ$ betrachtet, bleibt die mathematische Information über die Drehrichtung des Vektors erhalten. Du kannst nun eindeutig unterscheiden, ob eine Frequenz knapp über dem Träger (z. B. +100 kHz) oder knapp darunter (z. B. –100 kHz) liegt.

P03 – Gemini 2.0 Flash (27.12.2025)

Beschreibung: Erklärung des Belief-Propagation-Verfahrens, der benötigten Eingaben und der Verwendung des Decoders in Python. Grundlage für die eigene Implementierung und Tests.

Exakter Prompt:

```
Ich versuche die Python-Bibliothek bpdecoder für meinen LDPC-Code einzubauen.  
Wie funktioniert der Algorithmus dahinter und wie übergebe ich da meine  
LLR-Werte richtig?
```

Antwort der KI:

Die Python-Bibliothek `bpdecoder` (oft Teil von Paketen wie `ldpc`) ist ein essenzielles Werkzeug für die moderne Vorwärtsfehlerkorrektur (FEC). Sie implementiert den **Belief-Propagation-Algorithmus** (auch Sum-Product-Algorithmus genannt), der extrem nah an der Shannon-Grenze operiert.

Mathematisches Konzept (Tanner-Graph): Ein LDPC-Code (Low-Density Parity-Check) wird durch eine sehr dünn besetzte Parity-Check-Matrix $\mathbf{H}$ definiert. Diese Matrix kann als bipartiter Graph dargestellt werden:

- **Variable Nodes (Variablenknoten):** Repräsentieren die empfangenen Bits.
- **Check Nodes (Prüfknoten):** Repräsentieren die Paritätsgleichungen (jede Zeile in $\mathbf{H}$).

Der Algorithmus heisst "Belief Propagation", weil die Knoten iterativ ihre "Überzeugungen" (Beliefs) über den Zustand der Bits austauschen: 1. **Initialisierung:** Die Variablenknoten empfangen die intrinsische Information aus dem Kanal, üblicherweise als LLRs (Log-Likelihood Ratios). 2. **Message Passing:**

- Check-Nodes berechnen die Wahrscheinlichkeit, dass die Parity-Bedingung erfüllt ist, basierend auf den eingehenden Schätzungen.
- Variable-Nodes aktualisieren ihre LLRs basierend auf den Nachrichten der verbundenen Check-Nodes.

3. **Syndrom-Check:** Nach jeder Iteration wird hart entschieden (1 falls $\text{LLR} < 0$, sonst 0). Wenn das Syndrom $\mathbf{H} \cdot \mathbf{v} \equiv \mathbf{0} \pmod{2}$, ist das Codewort gültig.

Verwendung in Python: Hier ist eine Blaupause, wie du die Bibliothek in deine Pipeline einbaust:

```
import numpy as np
from ldpc import bp_decoder

# 1. Decoder-Instanz erzeugen (H_matrix muss vorgängig geladen werden)
# bp_method='ms' steht für Min-Sum, eine recheneffiziente
# Approximation des Sum-Product-Algorithmus.
decoder = bp_decoder(
    H_matrix,
    max_iter=50,
    bp_method='ms',
    channel_probs=None
)

# 2. Empfangene I/Q-Daten -> Demodulator -> LLRs berechnen
# llr_array ist ein 1D Numpy-Array mit float-Werten
decoded_bits = decoder.decode(llr_array)

# 3. Auswertung des Resultats
if decoder.converge:
    print(f"Decodierung erfolgreich!")
    # decoded_bits enthält nun das korrigierte Codewort
else:
    print("Maximale Iterationen erreicht - Decoding fehlgeschlagen.")
```

Die Eigenschaft `decoder.converge` ist für dein System äusserst wertvoll: Sie dient als verlässliche Bestätigung, dass die Payload fehlerfrei ist, was bei der Paket-Verifikation nach der Präambelsuche massiv hilft.

P04 – Gemini 2.0 Flash (24.03.2026)

Beschreibung: Analyse der Signalgenerierung und Hinweise auf Phasenoffset sowie fehlerhafte Berechnung der Phasenrotation. Unterstützung beim systematischen Debugging der

QPSK-Pipeline.

Exakter Prompt:

Bei meinem hybriden SDR-Setup funktioniert die LoRa-Erkennung bereits sehr gut, aber die extrahierten QPSK-Phasenwerte scheinen noch fehlerhaft zu sein. Woran könnte das liegen, wenn das De-Chirping an sich klappt?

Antwort der KI:

Dieses Fehlerbild – erfolgreiche LoRa-Detektion, aber fehlerhafte QPSK-Demodulation – ist ein klassisches Phänomen in hybriden CSS/PSK-Systemen. Der Grund liegt in der fundamental unterschiedlichen Sensibilität der beiden Modulationsarten gegenüber Phasen- und Frequenzfehlern.

LoRa (Chirp Spread Spectrum) ist extrem robust gegen Trägerfrequenzversatz (Carrier Frequency Offset, CFO). Ein CFO verschiebt lediglich den De-Chirp-Peak in der FFT um wenige Bins. QPSK hingegen ist eine absolute Phasenmodulation. Hier codiert der exakte Winkel (45° , 135° , 225° , 315°) die Daten.

Ursachenanalyse des fehlerhaften Phasenwerts:

1. **Carrier Frequency Offset (CFO) induzierte Phasenrotation:** Ein verbleibender Frequenzversatz Δf zwischen Sender und Empfänger dreht die Konstellationspunkte kontinuierlich. Die Phase über die Zeit t ändert sich gemäss:

$$\phi_{error}(t) = 2\pi \cdot \Delta f \cdot t$$

Da sich diese Phase während der Integration über das Symbol aufkumuliert, "verwischen" die QPSK-Punkte zu einem Ring im I/Q-Diagramm, oder sie rotieren in den falschen Quadranten. Selbst wenn der CFO für LoRa unbedeutend klein ist (z. B. 500 Hz), führt er bei langen Symbolauern zu Phasenfehlern von über 180° .

2. **Timing-Offset (Fractional Symbol Timing):** Die LoRa-Präambel-Detektion liefert das Start-Timing auf Sample- oder Bin-Ebene. Ist dieser Startpunkt auch nur um wenige Sub-Samples verschoben, führt dies im Frequenzbereich (nach der FFT) zu einem *linearen Phasengradienten*. Laut den Eigenschaften der Fouriertransformation gilt:

$$\mathcal{F}\{x(t - \tau)\} = X(f) \cdot e^{-j2\pi f\tau}$$

Ein Timing-Fehler τ erzeugt also einen Phasen-Offset, der abhängig von der Frequenz (und damit vom Chirp) ist.

3. **Fehlerhafte De-Chirp Referenz (Phase Continuity):** Wenn dein lokaler Down-Chirp, den du für die Multiplikation generierst, nicht phasenkontinuierlich ist (die Phase springt an den Symbolgrenzen), wird dieser Phasensprung direkt auf dein demoduliertes QPSK-Symbol übertragen.

Systematisches Debugging-Vorgehen:

- **Schritt 1 (Visualisierung):** Plote das Scatter-Diagramm (I vs. Q) der QPSK-Werte *nach* der Demodulation. Wenn du einen sauberen Ring siehst, ist es ein unkorrigierter CFO. Wenn du vier verschmierte Wolken siehst, ist es Rauschen/Timing.

- **Schritt 2 (Differenziell):** Ändere die Übertragung testweise auf DQPSK (Differential QPSK). Dabei wird die Information in der *Änderung* der Phase von Symbol zu Symbol codiert. Langsame CFO-Rotationen kürzen sich hierbei heraus ($\Delta\phi = (\phi_n + \phi_{err}) - (\phi_{n-1} + \phi_{err})$). Funktioniert DQPSK einwandfrei, weisst du mit 100%-iger Sicherheit, dass es ein CFO-Problem ist.
-

P05 – Gemini 2.0 Flash (29.03.2026)

Beschreibung: Vorgehen zur Laufzeitmessung der SDR-Pufferverarbeitung, zum Vergleich mit der verfügbaren Bufferdauer und zur Erkennung möglicher Überläufe. Grundlage für die eigene Latenzanalyse.

Exakter Prompt:

Ich habe die Befürchtung, dass mein SDR-Skript in Python zu langsam ist.
Wie baue ich am besten eine präzise Laufzeitmessung ein, um herauszufinden, ob mein Puffer voll läuft und ich Samples verliere?

Antwort der KI:

Die Messung der SDR-Pufferverarbeitung ist kritisch, um *Buffer Overruns* (Überläufe) und den damit verbundenen Verlust von Basisband-Samples zu verhindern.

Das Kernkonzept ist der Vergleich zwischen der **Pufferdauer** (wie lange das SDR braucht, um den Block zu füllen) und der **Rechenzeit** (wie lange dein PC braucht, um den Block zu verarbeiten).

1. Berechnung der verfügbaren Bufferdauer:

$$T_{buffer} = \frac{N_{samples}}{f_s}$$

Bei einer Abtastrate (f_s) von z. B. 2 MSPS und einer Blockgrösse von 4096 Samples hat dein System exakt **2.048 ms** Zeit.

2. Vorgehen zur Laufzeitmessung in Python: Verwende immer `time.perf_counter()`, da dieser Timer die höchste Hardware-Präzision bietet:

```
import time
import numpy as np

buffer_size = 4096
sample_rate = 2e6
max_time = buffer_size / sample_rate

overrun_count = 0
runtimes = []

while streaming:
    # 1. Blockierenden Lese-Aufruf starten
```

```

buf.refill()
data = buf.read()

# 2. Hochpräzise Zeitmessung starten
t_start = time.perf_counter()

# --- DEINE SIGNALVERARBEITUNG HIER ---
# z.B. complex_data = np.frombuffer(...)
# fft_res = np.fft.fft(complex_data)
# -----

# 3. Zeitmessung stoppen
t_end = time.perf_counter()
duration = t_end - t_start
runtimes.append(duration)

if duration > max_time:
    overrun_count += 1
    print(f"WARNUNG: Deadline verfehlt! Dauer: {duration*1000:.2f} ms")

print(f"Durchschnittliche Latenz: {np.mean(runtimes)*1000:.2f} ms")
print(f"Anzahl Overruns: {overrun_count}")

```

Erkennung von Überläufen: Wenn dein Skript chronisch langsamer ist ($duration > T_{buffer}$), wird der FIFO-Puffer in der libiio Kernel-Treiberschicht volllaufen. Die libiio C-Bibliothek meldet in diesem Moment meist einen stillen Drop, oder der Datenstrom wird diskontinuierlich, was sofort zu Phasensprüngen im Signal führt.

P06 – Claude Sonnet 4.6 (02.04.2026)

Beschreibung: Vorschläge für die Struktur der Hardwaremodule und für Multiplikations- und Signalverarbeitungslogik. Daraus wurde die Grundstruktur der Verilog-Module aufgebaut.

Exakter Prompt:

Wie würdest du meine Python-Logik (De-Chirp Multiplikation, FFT, Argmax) am besten in strukturierte Verilog-Module für ein FPGA-Design umschreiben? Worauf muss ich achten?

Antwort der KI:

Die Übertragung einer Python-basierten LoRa/QPSK-Logik in strukturierte Verilog-Module für FPGAs erfordert ein Umdenken von serieller zu stark parallelisierter, takt-basierter Register-Transfer-Logik (RTL).

Vorgeschlagene Modul-Architektur:

1. **Input / AXI-Stream Wrapper:** Ein Modul, das die I/Q-Daten vom ADC via AXI-Stream (tdata, tvalid, tready) asynchron in die System-Clock-Domäne übernimmt.

2. **DDS (Direct Digital Synthesis) / NCO:** Zur Generierung des lokalen Up- und Down-Chirps für die De-Chirp-Multiplikation. Nutze hierfür einen Phasen-Akkumulator, der seinen Schrittweiten-Wert taktweise erhöht.

3. **Komplexer Multiplizierer (De-Chirp):** Ein eingehendes komplexes Sample ($I_{in} + jQ_{in}$) muss mit dem lokalen Chirp ($I_{lo} + jQ_{lo}$) multipliziert werden:

$$\bullet I_{out} = I_{in} \cdot I_{lo} - Q_{in} \cdot Q_{lo}$$

$$\bullet Q_{out} = I_{in} \cdot Q_{lo} + Q_{in} \cdot I_{lo}$$

Tipp: Nutze dafür die FPGA-internen DSP48-Slices, instanziiere sie als Pipelined-Multiplier (3-4 Taktzyklen Latenz), um den kritischen Pfad zu verkürzen.

4. **FFT-Core (Xilinx IP):** Füttere die De-Chirped Samples in den fertigen Xilinx FFT IP-Core (konfiguriert auf Pipelined, Streaming I/O).

5. **ArgMax / Peak Detection:** Ein einfaches Modul, das das Betragsquadrat $I^2 + Q^2$ der FFT-Ausgänge berechnet und kontinuierlich den Index des Bins mit dem höchsten Wert speichert.

Wichtig für Verilog: Vermeide Divisionen! Multiplikationen mit fixed-point Brüchen oder Shift-Operationen (Bit-Shifting) sind essentiell.

P07 – Claude Sonnet 4.6 (05.04.2026)

Beschreibung: Vorschläge zur strukturellen Verbindung der Module mit dem vorhandenen System-IP und zur Organisation des Blockdesigns. Unterstützung bei der Systemintegration in Vivado.

Exakter Prompt:

Ich habe jetzt meine eigenen Verilog-Module geschrieben. Wie packe ich die in Vivado in ein Blockdesign und verbinde sie per AXI-Stream mit dem bestehenden ADC-Controller?

Antwort der KI:

Um deine selbst geschriebenen Verilog-Module nahtlos in das Vivado-Referenzdesign (Block Design, z.B. für Zynq-7000/UltraScale) einzubinden, solltest du sie in Xilinx IP-Cores verpacken.

Vorschlag zur Organisation und Einbindung:

- **AXI-Stream als Standard:** Stelle sicher, dass alle Datenpfade (Samples rein, FFT raus) strikt das AXI4-Stream-Protokoll verwenden. Deine Module benötigen mindestens `m_axis_tdata` und `m_axis_tvalid`. `tready` ist wichtig, um *Backpressure* zu verarbeiten, wenn nachfolgende Blöcke (wie der DMA-Controller) ausgelastet sind.
- **IP Packager nutzen:** 1. Erstelle ein neues Verzeichnis für deinen Core. 2. Nutze "Tools -> Create and Package New IP". 3. Importiere deine Verilog-Dateien. 4. Im Tab "Ports and

Interfaces" weist Vivado automatisch den Signalen (z.B. `s_axis_tvalid`) die korrekten AXI-Klassen zu.

- **Einbindung im Block Design:** Füge deinen neuen IP-Core in das Canvas ein. Der ADC-Ausgang des SDR-Referenzdesigns (z.B. der `axi_ad9361` Core) liefert einen kontinuierlichen AXI-Stream. Verbinde diesen mit dem Eingang deines De-Chirp-Moduls.
- **Konfiguration via AXI4-Lite:** Für Kontrollregister (Schwellwerte, Sync-Words, CFO-Korrekturfaktoren) solltest du einen AXI4-Lite Slave-Port vorsehen. Dadurch kann der Linux-Prozessor (Zynq PS) dynamisch aus einem Python-Skript (über `/dev/mem`) Parameter deines Hardware-Moduls zur Laufzeit verändern.

Lass Vivado am Ende die "Connection Automation" durchführen, um Takt (`ac1k`) und Reset (`aresetn`) an die globale System-Clock anzubinden.

P08 – Gemini 2.5 Pro (13.04.2026)

Beschreibung: Übersetzung und technische Erklärung relevanter Passagen zu Anschlüssen, Netzwerk und Hardware. Die Angaben wurden mit der Originaldokumentation verglichen.

Exakter Prompt:

Ich arbeite mit der Dokumentation meines SDR-Boards, aber einige technische Begriffe sind schwer verständlich. Welche Parameter (wie IP, Stromversorgung, Boot-Jumper) sind für die erste Inbetriebnahme essenziell?

Antwort der KI:

Bei der Inbetriebnahme chinesischer SDR-Boards (häufig Klone der ADALM-Pluto oder PicoZed-Architekturen) muss man die maschinell übersetzten oder originalen Dokumentationen genau analysieren.

Relevante Parameter aus der Dokumentation:

1. **Netzwerk- und IP-Konfiguration:** Sehr oft wird ein virtuelles USB-Ethernet-Interface (RNDIS/CDC) aufgespannt. Die Default-IP des SDRs ist meist `192.168.2.1` oder `192.168.1.10`. Die Dokumentation definiert auch die Standard-Credentials für SSH (meist `root / analog`).
2. **Jumper- und Boot-Switch-Einstellungen:** Wie bootet das FPGA? Achte auf Tabellen, die die Stellung von DIP-Switches oder Jumpers beschreiben. `MODE[2:0]` Pins bestimmen, ob von SD-Karte, QSPI-Flash oder JTAG gebootet wird. Falsche Stellungen führen zu einem "toten" Board ohne LED-Aktivität.
3. **Stromversorgung (Power Constraints):** Ein AD9361 Transceiver gepaart mit einem Zynq-FPGA kann im Sendebetrieb über 1.5 Ampere ziehen. Die Dokumentation warnt oft davor, nur USB zur Versorgung zu nutzen (Spannungsabfall). Es ist dringend erforderlich, den spezifizierten externen 5V/2A DC-Anschluss zu nutzen, um Brown-Out-Resets unter Last zu vermeiden.
4. **HF-Spezifikation (SMA Ports):** Einige Boards nutzen interne RF-Switches für TDD (Time Division Duplex), was bedeutet, dass TX und RX sich einen SMA-Port teilen. Prüfe

im Schema, ob Port A zwingend RX und Port B zwingend TX ist, oder ob es umschaltbar ist (oft als TRX bezeichnet).

P09 – Claude Sonnet 4.6 (13.04.2026)

Beschreibung: Vorschläge für die Netzwerkkonfiguration und Fehlersuche. Die schlussendlich verwendete Lösung wurde selbst getestet und für die eigene Hardware angepasst.

Exakter Prompt:

Mein SDR hat die IP 192.168.2.1. Wie richte ich unter Linux am besten das USB-Ethernet-Interface ein, sodass die Route stabil bestehen bleibt und nicht vom NetworkManager überschrieben wird?

Antwort der KI:

Um unter Linux eine stabile Ethernet-Verbindung (oft über ein virtuelles USB-Ethernet-Interface oder einen dedizierten RJ45-Port) zum SDR herzustellen, bedarf es korrekter Subnetz-Routings.

1. Identifikation der Schnittstelle: Stecke das SDR ein und führe `dmesg | tail -n 20` oder `ip link` aus. Typischerweise taucht ein Interface wie `eth1`, `enx00...` oder `usb0` auf.

2. Temporäre IP-Zuweisung: Angenommen, die Doku sagt, das SDR hat 192.168.2.1. Dein Rechner muss im selben Subnetz liegen (aber mit anderer Endung, z.B. .10):

```
sudo ip addr add 192.168.2.10/24 dev usb0
sudo ip link set usb0 up
```

3. Testen der Verbindung: Prüfe Layer 3 (IP) mit `ping 192.168.2.1`. Wenn das klappt, teste, ob die `iiod` (IIO-Daemon) Software auf dem SDR läuft. Dazu benutzt du das Tool `iio_info`:

```
iio_info -u ip:192.168.2.1
```

Dies sollte eine lange Liste von Hardware-Attributen (ADC, DAC, LOs) ausgeben.

4. Fehlersuche (NetworkManager Konflikte): Linux-Desktop-Systeme verwenden den `NetworkManager`, der manuell gesetzte IPs oft sofort wieder überschreibt. Falls `ping` nur sporadisch funktioniert, deaktiviere das Management für dieses Interface: Erstelle eine Datei in `/etc/NetworkManager/conf.d/99-unm`

```
[keyfile]
unmanaged-devices=interface-name:usb0
```

Starten den Service mit `sudo systemctl restart NetworkManager` neu.

P10 – Gemini 2.5 Pro (13.04.2026)

Beschreibung: Beispielcode für eine `libiio`-Verbindung und das Auslesen von Daten. Als Grundlage für den eigenen Hardware-Test angepasst.

Exakter Prompt:

Kannst du mir einen sauberen Python-Snippet mit libiio schreiben? Ich will einfach nur verbinden, den AD9361 auf 868 MHz stellen und prüfen, ob ich einen IQ-Puffer mit 4096 Samples ausgelesen bekomme.

Antwort der KI:

Um mit Python und der `libiio` Daten vom SDR auszulesen, muss die Pipeline aus Context, Device, Channel und Buffer korrekt instanziiert werden.

Hier ist ein robuster Beispielcode, der den Hardware-Zugriff initialisiert und einen Puffer ausliest:

```
import iio
import numpy as np

# 1. Verbindung zum SDR herstellen
URI = "ip:192.168.2.1"
try:
    ctx = iio.Context(URI)
except Exception as e:
    print(f"Fehler bei Verbindung: {e}")
    exit(1)

# 2. RX-Frontend (Phy) konfigurieren (z.B. AD9361)
phy = ctx.find_device("ad9361-phy")
phy.attrs["voltage0_rf_bandwidth"].value = str(int(2e6))    # 2 MHz BW
phy.channels[0].attrs["frequency"].value = str(int(868e6))  # LO 868 MHz

# 3. ADC (Data-Device) initialisieren
rx_adc = ctx.find_device("cf-ad9361-lpc")
rx_i = rx_adc.find_channel("voltage0", False)
rx_q = rx_adc.find_channel("voltage1", False)
rx_i.enabled = True
rx_q.enabled = True

# 4. Puffer erstellen (z.B. 4096 Samples pro Block)
buffer_size = 4096
rx_buf = iio.Buffer(rx_adc, buffer_size, False)

# 5. Daten lesen
rx_buf.refill()
data = rx_buf.read()

# 6. Wandlung von Roh-Bytes (16-Bit signed) in IQ-Arrays
raw_data = np.frombuffer(data, dtype=np.int16)
# I/Q sind interleavend: [I, Q, I, Q, ...]
i_samples = raw_data[0::2]
q_samples = raw_data[1::2]
complex_samples = i_samples + 1j * q_samples
```

```
print(f"Gelesen: {len(complex_samples)} komplexe Samples")
```

P11 – Gemini 2.5 Pro (14.04.2026)

Beschreibung: Verschiedene Ansätze zur Netzwerkkonfiguration und Diagnose der Linux-Umgebung. Die letztlich verwendete Methode wurde selbst überprüft.

Exakter Prompt:

Nach einem Neustart des SDR-Boards werden die gesetzten IP-Einstellungen leider zurückgesetzt. Woran liegt das bei diesem Embedded-Linux und wie kann ich die Konfiguration für meine Tests dauerhaft speichern?

Antwort der KI:

Das Phänomen, dass das SDR-Board nach einem Reboot (Power-Cycle) seine Netzwerkeinstellungen "vergisst", ist bei Embedded-Linux-Systemen (wie Buildroot oder PetaLinux) extrem häufig.

Diagnose der Umgebung: Die meisten SDR-Images booten das Root-Dateisystem (/) als flüchtiges `initramfs` (in den Arbeitsspeicher). Alle Änderungen in `/etc/network/interfaces` oder `/etc/ipaddr.txt` existieren nur im RAM und sind beim Neustart weg.

Ansätze zur zuverlässigen Konfiguration:

- **Ansatz 1: Persistente Partition nutzen (Empfohlen)** Oft ist die FAT32-Boot-Partition (auf der `BOOT.BIN` liegt) beim Boot-Vorgang eingebunden. Einige Systeme lesen beim Start eine Skriptdatei (z.B. `uEnv.txt` oder ein `network_config` File) von dieser SD-Karten-Partition. Prüfe in der Konsole mit `mount`, ob `/media/sd-mmcblk0p1` existiert und lege dort eine Konfiguration ab.
 - **Ansatz 2: Start-Skript über USB-UART (Automatisierung)** Für Testläufe kannst du ein Python-Skript am Host-PC schreiben, das sich via serieller Schnittstelle (z.B. `/dev/ttyUSB0`) einloggt und den Befehl absetzt: `ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up` Mit der Bibliothek `pyserial` lässt sich das in 10 Zeilen vollautomatisieren.
 - **Ansatz 3: Kernel-Boot-Parameter anpassen** Bist du mutig, veränderst du die `bootargs` in der U-Boot Umgebung. Dort kann man das Netzwerk hart kodieren: `ip=192.168.1.10::192.168.1.1`
-

P12 – Gemini 2.5 Pro (16.04.2026)

Beschreibung: Python-Grundstruktur zur Initialisierung des SDR, FFT-Auswertung und Darstellung der Messdaten. Der Code wurde angepasst und anschliessend selbst validiert.

Exakter Prompt:

Ich brauche einen simplen Python-Spektrumanalysator für mein SDR. Das Ganze sollte eine FFT, einen Live-Plot, Wasserfall und Max-Hold haben. Wie strukturiere ich das am besten, ohne dass die GUI einfriert?

Antwort der KI:

Ein performanter Python-Spektrumanalysator erfordert die strikte Trennung von Datenakquise (Hardware) und Visualisierung (GUI). Da du mit `pyadi-iiio` (`adi`) arbeitest, können wir die Daten extrem performant abgreifen, indem wir eine *Matrix* aus mehreren FFT-Chunks über Numpy vektorisieren.

Architektur-Vorschlag (Burst-Catcher):

1. **Hardware-Setup:** Wir setzen den `rx_buffer_size` gross genug, um z.B. 40 FFT-Fenster auf einmal zu fangen.
2. **Vektorisierte FFT:** Wir formen den 1D-Puffer mittels `reshape` in eine Matrix um. `np.fft.fft` berechnet dann alle 40 Chunks gleichzeitig ohne lahme Python-Schleife.
3. **Visualisierung (matplotlib):** Wir aktualisieren Plot-Objekte (`set_ydata`) anstatt den Plot neu zu zeichnen (`plt.pause()`).

Hier ist dein einsatzbereiter Python-Code, optimiert für den ADALM-Pluto:

```
import numpy as np
import adi
import matplotlib.pyplot as plt
import warnings

warnings.filterwarnings('ignore')

SDR_IP = "ip:192.168.1.10"
CENTER_FREQ = int(2450000000) # 2.4 GHz
SAMPLERATE = int(10000000)    # 10 MHz
BANDWIDTH = int(10000000)

FFT_SIZE = 1024
FFT_CHUNKS = 40
BUFFER_SIZE = FFT_SIZE * FFT_CHUNKS
WATERFALL_ROWS = 100

print(f"Verbinde mit SDR auf {SDR_IP}...")
sdr = adi.Pluto(SDR_IP)
sdr.sample_rate = SAMPLERATE
sdr.rx_lo = CENTER_FREQ
sdr.rx_rf_bandwidth = BANDWIDTH
sdr.rx_buffer_size = BUFFER_SIZE
sdr.gain_control_mode_chan0 = "manual"
sdr.rx_hardwaregain_chan0 = 50
sdr.rx()

plt.ion()
fig, (ax_spec, ax_water) = plt.subplots(2, 1, figsize=(12, 9))
```

```

fig.patch.set_facecolor('#1e1e1e')

freqs_mhz = np.fft.fftshift(np.fft.fftfreq(FFT_SIZE, 1/SAMPLERATE)) / 1e6

ax_spec.set_facecolor('#1e1e1e')
line, = ax_spec.plot(freqs_mhz, np.full(FFT_SIZE, -100), color='#00ffcc')
line_max, = ax_spec.plot(freqs_mhz, np.full(FFT_SIZE, -100), color='#ff0044')

waterfall_data = np.full((WATERFALL_ROWS, FFT_SIZE), -90.0)
img = ax_water.imshow(waterfall_data, aspect='auto', cmap='inferno', vmin=-80, vmax=-20)
plt.tight_layout()

window = np.hanning(FFT_SIZE)
max_hold_data = np.full(FFT_SIZE, -120.0)

try:
    while plt.fignum_exists(fig.number):
        rx_data = sdr.rx()
        chunks = rx_data[:FFT_SIZE * FFT_CHUNKS].reshape((FFT_CHUNKS, FFT_SIZE)) / 2048.0

        chunks_windowed = chunks * window
        fft_complex = np.fft.fft(chunks_windowed, axis=1) / FFT_SIZE
        fft_shifted = np.fft.fftshift(fft_complex, axes=1)

        psd_db_matrix = 10 * np.log10(np.abs(fft_shifted)**2 + 1e-12)
        psd_db_live = np.max(psd_db_matrix, axis=0)
        max_hold_data = np.maximum(max_hold_data, psd_db_live)

        line.set_ydata(psd_db_live)
        line_max.set_ydata(max_hold_data)

        waterfall_data = np.roll(waterfall_data, -1, axis=0)
        waterfall_data[-1, :] = psd_db_live
        img.set_data(waterfall_data)
        plt.pause(0.01)
except KeyboardInterrupt:
    pass

```

Erklärung zur Darstellung:

- **Spektrum (Live & Max-Hold):** Wir halten einen kumulierten Maximalwert (`max_hold_data`), der durch `np.maximum` ständig überschrieben wird, sobald ein stärkerer Peak auftaucht. Ideal, um flüchtige Bursts zu erfassen.
- **Wasserfall:** Mit `np.roll(waterfall_data, -1, axis=0)` scrollen wir die Matrix nach oben und schreiben die neue Zeile ans untere Ende. `imshow()` rendert diese Matrix als Heatmap.

P13 – Claude Sonnet 4.6 (30.04.2026)

Beschreibung: Vorschlag für Peak-Suche und Berechnung des Frequenzversatzes. Grundlage für die eigene CFO-Analyse und anschliessend an die konkrete Signalverarbeitung angepasst.

Exakter Prompt:

Wenn ich mein de-chirptes Signal per FFT anschau, wie berechne ich aus der Position des Peaks den exakten Carrier Frequency Offset (CFO) in Hertz?

Antwort der KI:

Die Messung der Frequenzabweichung (Carrier Frequency Offset, CFO) mittels FFT beruht auf der Lokalisierung des spektralen Peaks eines bekannten Sendesignals. Bei LoRa-Systemen nutzt man dafür oft die reinen Chirps der Präambel. Nach einem De-Chirp-Schritt resultieren diese in einem konstanten Ton (Peak) in der FFT.

Berechnung des Grob-Versatzes (Integer-CFO): Nach der FFT ($X = \text{fftshift}(\text{fft}(x))$) des empfangenen (und de-chirpten) Basisbandsignals suchst du den Index k_{peak} mit maximaler Energie:

$$k_{peak} = \text{argmax}(|X|)$$

Der Index k_{center} entspricht der Frequenz 0 Hz (DC) und liegt bei $N/2$ (wobei N die FFT-Grösse ist). Der Frequenzversatz in Hertz berechnet sich zu:

$$\Delta f_{coarse} = (k_{peak} - \frac{N}{2}) \cdot \frac{f_s}{N}$$

wobei f_s die Abtastrate in Hz ist.

Sub-Bin-Interpolation für hohe Präzision (Fein-CFO): Da der echte Peak meist *zwischen* zwei diskreten FFT-Bins liegt, nutzt man Parabel-Interpolation der drei höchsten Bins ($\alpha = |X[k-1]|, \beta = |X[k]|, \gamma = |X[k+1]|$):

$$\Delta k_{fine} = \frac{1}{2} \cdot \frac{\alpha - \gamma}{\alpha - 2\beta + \gamma}$$

Der exakte Frequenzversatz ist dann:

$$\Delta f_{exact} = (k_{peak} + \Delta k_{fine} - \frac{N}{2}) \cdot \frac{f_s}{N}$$

Alternative: CFO aus Phasenrotation der Präambel Statt einer Interpolation kannst du den feinen CFO über die Phasenänderung zwischen den aufeinanderfolgenden Präambel-Chirps bestimmen. Jeder Chirp liefert an seinem Peak einen komplexen Phasenwinkel. Die Phasendifferenz $\Delta\phi_{Mittel}$ zwischen zwei Symbolen entspricht der Phasendrift:

$$\Delta f_{CFO} = \frac{\Delta\phi_{Mittel}}{2\pi \cdot T_{Symbol}}$$

Dieser Wert ist extrem präzise und dient in deinem hybriden Modem zur Korrektur vor der DQPSK-Demodulation.

P14 – Gemini 2.5 Pro (02.05.2026)

Beschreibung: Ansätze zur Bestimmung der zeitlichen Verschiebung aus den detektierten FFT-Bins und zur Korrektur des Empfangsfensters. Unterstützung beim Debugging der zeitlichen Synchronisation.

Exakter Prompt:

Wie kriege ich aus der detektierten LoRa-Präambel (dem FFT-Bin) berechnet, um wie viele Samples mein Empfangsfenster verschoben ist, damit ich die nachfolgenden Symbole exakt treffe?

Antwort der KI:

Bei LoRa nutzt man die Eigenschaft des Chirp-Signals, dass Zeit und Frequenz fest miteinander gekoppelt sind, um eine exakte Symbolsynchronisation durchzuführen.

Die Präambel besteht aus mehreren reinen, identischen Up-Chirps. Wenn der Empfänger sein lokales "De-Chirp" Signal (ein Down-Chirp) generiert und mit dem empfangenen Up-Chirp multipliziert, heben sich die linearen Frequenzänderungen auf. Das Resultat ist ein konstanter Ton.

Bestimmung des zeitlichen Offsets: Wenn dein Empfangsfenster exakt mit dem eintreffenden Chirp synchron ist, liegt der Frequenz-Peak des de-chirped Signals bei 0 Hz (also FFT-Bin 0). Kommt das Signal jedoch später (zeitlicher Offset Δt) an, verschiebt sich die Frequenz des resultierenden Tons linear zur Zeitverschiebung.

Der detektierte Peak bei Bin k_{peak} (nach FFT-Shift bezogen auf 0) liefert dir den Versatz in Samples:

$$\Delta n_{offset} = k_{peak} \cdot \frac{N_{samples_per_symbol}}{N_{FFT}}$$

(Hierbei ist $N_{samples_per_symbol}$ die Anzahl Samples pro Symbol. Meistens wählt man $N_{FFT} = N_{samples_per_symbol}$, dann ist die Verschiebung in Bins exakt identisch mit der Verschiebung in Samples.)

Korrektur des Empfangsfensters: Um die nachfolgenden Payload-Symbole perfekt auszurichten, verschiebst du deinen Lese-Pointer im I/Q-Puffer um exakt Δn_{offset} Samples zurück (oder nach vorne). Ein Fehler in dieser Berechnung führt direkt zu fehlerhafter Symbolauswertung (die decodierten Bins in der LoRa-Payload haben ein systematisches Offset) und zu Inter-Symbol-Interferenz (ISI).

P15 – Gemini 2.5 Pro (03.05.2026)

Beschreibung: Herleitung und Implementierungsansatz für eine Phasen-Rückrotation anhand des geschätzten Frequenzoffsets. Als Grundlage für die eigene Implementierung verwendet.

Exakter Prompt:

In meinem Konstellationsdiagramm rotieren die QPSK-Punkte kontinuierlich, was auf einen Frequenzversatz hindeutet. Wie implementiere ich in Numpy am besten eine Phasen-Rückrotation, um diesen restlichen CFO zu kompensieren?

Antwort der KI:

Ein verbleibender Trägerfrequenzversatz (Carrier Frequency Offset, CFO) bewirkt, dass sich die Phase der Basisbandsamples kontinuierlich über die Zeit dreht. In der Formel:

$$y(t) = x(t) \cdot e^{j2\pi\Delta f_{CFO}t}$$

Für QPSK ist das verheerend, da die Information im Absolutwinkel steckt. Wenn sich die Konstellation im I/Q-Diagramm dreht, werden die Symbole in die falschen Quadranten decodiert.

Herleitung der Kompensation: Um das ursprüngliche Signal $x(t)$ zurückzuerhalten, müssen wir das empfangene Signal $y(t)$ mit der konjugiert-komplexen Phasenrotation multiplizieren:

$$\hat{x}(t) = y(t) \cdot e^{-j2\pi\Delta f_{CFO}t}$$

Implementierungsansatz (Python/Numpy): Angenommen, du hast den f_{CFO} (in Hz) berechnet und deine Abtastrate f_s ist bekannt. Der Array-Index n entspricht der Zeit $t = n \cdot T_s = n/f_s$.

```
import numpy as np

# n ist ein Array mit den Sample-Indizes [0, 1, 2, ..., N-1]
n_indices = np.arange(len(received_iq))

# Berechne den Korrekturwinkel für jedes Sample
# theta = -2 * pi * f_cfo * t
theta_corr = -2.0 * np.pi * f_cfo * (n_indices / sample_rate)

# Erstelle den komplexen Zeiger
correction_phasor = np.exp(1j * theta_corr)

# Phasen-Rückrotation durchführen
corrected_iq = received_iq * correction_phasor
```

Nach dieser Multiplikation sind die QPSK-Punkte wieder stabil in ihren Ursprungsquadranten eingefroren.

P16 – Gemini 2.5 Pro (09.05.2026)

Beschreibung: Vorschläge für Tests ohne Antenne, Varianten mit und ohne CFO-Korrektur sowie zur Trennung von Hardware-, Software- und Interferenzursachen. Unterstützung bei der Fehlereingrenzung.

Exakter Prompt:

Mein Skript detektiert gelegentlich LoRa-Präambeln, auch wenn der Sender gar nicht aktiv ist. Wie kann ich systematisch isolieren, ob dies an externem Rauschen, USB-Störungen oder an einem zu niedrig gewählten Schwellwert liegt?

Antwort der KI:

Phantom-Signale (False Positives) und falsche Präambel-Erkennungen können den Empfänger komplett blockieren, da er Ressourcen verschwendet, um Rauschen zu decodieren. Die Fehlersuche erfordert eine strikte Isolation der Einflüsse.

1. Trennung von Hardware- und Interferenzursachen:

- **Der 50-Ohm-Test:** Schraube die Antenne ab und terminiere den SMA-Port mit einem 50-Ohm Widerstand. Damit empfangst du kein RF-Signal von aussen.
- Wenn die Phantom-Signale *verschwinden*: Es handelt sich um echte Funk-Interferenzen (z.B. andere Sender im ISM-Band) oder atmosphärisches Rauschen.
- Wenn die Phantom-Signale *bleiben*: Das Problem entsteht intern. Oft ist es ein starker DC-Offset (LO-Leakage in der Mitte des Spektrums) des SDR-Chips oder digitales Rauschen vom USB-Bus.

2. Softwareseitige Analyse der Präambelerkennung:

- **Schwellwert-Test (Thresholding):** Betrachte den Ausgang deines Kreuzkorrelators. Liegt dein Threshold-Wert nur knapp über dem Noise-Floor, werden Rauschspitzen als Paket-Anfang gewertet. Hebe den Threshold testweise stark an.
- **CFO-Einfluss:** Deaktiviere deine automatische CFO-Korrektur. Wenn ein falscher Peak detektiert wird, könnte die nachfolgende CFO-Kompensation das Rauschen so verdrehen, dass es wie ein Signal aussieht.
- **Zeitliche Validierung:** Eine echte LoRa-Präambel besteht aus z.B. 8 exakt hintereinander folgenden, gleich starken Peaks nach dem De-Chirp. Schreibe einen Plausibilitäts-Check: Ein Paket gilt nur dann als erkannt, wenn der Peak-Abstand in exakt N Samples wieder auftritt. Ein zufälliges Phantom-Signal erfüllt diese strenge zeitliche Korrelation nicht.

P17 – Gemini 2.5 Pro (09.05.2026)

Beschreibung: Überlegungen zur Rechenlast der Korrelation, zu Puffern und zur Echtzeitverarbeitung. Die Erkenntnisse flossen in die Untersuchung der Softwarelatenz ein.

Exakter Prompt:

Die Kreuzkorrelation im Zeitbereich verursacht aktuell eine sehr hohe CPU-Auslastung. Wie kann ich die Präambel-Suche effizienter gestalten, zum Beispiel über FFT oder Overlap-and-Save, um keine Samples zu verpassen?

Antwort der KI:

Die Kreuzkorrelation zur Präambelerkennung ist der grösste Flaschenhals im Zeitbudget eines SDR-Empfängers. Eine klassische Zeitbereichskorrelation benötigt $\mathcal{O}(N^2)$ Operationen, was bei Abtastraten von > 1 MHz moderne CPUs in Echtzeit überlastet.

1. Transformation in den Frequenzbereich (Fast Convolution): Die wichtigste Optimierung ist die Nutzung des Faltungstheorems. Anstatt im Zeitbereich zu falten, multiplizierst du im Frequenzbereich.

- $FFT_{signal} = FFT(Signal)$
- $FFT_{preamble} = FFT(Preamble_{Referenz}^*)$ (konjugiert komplex)
- $Korrelation_{zeit} = IFFT(FFT_{signal} \cdot FFT_{preamble})$

Dies reduziert die Komplexität auf $\mathcal{O}(N \log N)$ und macht Echtzeitverarbeitung überhaupt erst möglich.

2. Dezimierung (Downsampling) in der Suchphase: Für das reine Detektieren *ob* ein Paket anfängt, benötigst du nicht die volle Auflösung von z.B. 2 MSPS. Schalte ein FIR-Tiefpassfilter vor und dezimiere das Signal um den Faktor 4 (auf 500 kSPS). Führe die Kreuzkorrelation auf diesem geschrumpften Puffer durch. Erst wenn der Schwellwert überschritten wird, springst du in den hochauflösenden Puffer, um das exakte Timing zu berechnen.

3. Overlap-and-Save Methode: Da das I/Q-Signal in Blöcken aus dem SDR kommt, kann eine Präambel genau auf der Grenze zwischen Block i und Block $i + 1$ liegen. Du musst zwingend einen Ringpuffer nutzen, bei dem du die letzten $M - 1$ Samples des alten Blocks an den neuen Block anhängst (M ist die Länge der Präambel). Nur so stellst du sicher, dass keine Pakete an den Puffer-Grenzen abgeschnitten werden.

P18 – Gemini 2.5 Pro (14.05.2026)

Beschreibung: Ansatz für feste Sende- und Empfangsslots und die Reservierung eines definierten Anteils der Slot-Zeit für die Decodierung. Grundlage für die eigene TDMA-Implementierung.

Exakter Prompt:

Mein Python-Code braucht zu lange. Wie baue ich am besten eine TDMA-Struktur, bei der das SDR nur ganz kurz aufnimmt und die Software dann Pause für die FFT und LDPC-Rechnerei hat?

Antwort der KI:

Ein Time Division Multiple Access (TDMA) Schema ist essentiell, wenn die Signalverarbeitung (Demodulation/LDPC) länger dauert als das Empfangsfenster selbst. Wenn du den SDR nicht pausierst, führt die hohe Rechenlast zum unweigerlichen Pufferüberlauf.

Das Prinzip der strikten Slot-Zuweisung: Die Zeit wird in fixe Frames (z.B. 200 ms) unterteilt. Beide SDR-Knoten müssen die exakte Rahmenstruktur kennen (z.B. durch GPS-Synchronisation). Ein Frame besteht aus:

1. **TX-Slot (Knoten A) / RX-Slot (Knoten B):** 0 – 20 ms. Hier ist der Sender aktiv. Der Empfänger nimmt in dieser Zeit *nur* rohe I/Q-Samples auf und schiebt sie unberührt in den Arbeitsspeicher (RAM).
2. **Verarbeitungs-Slot (Knoten B):** 20 – 100 ms. Der SDR-Empfang wird gestoppt (!). Die CPU nutzt 100% ihrer Rechenleistung, um den RAM-Puffer aus dem RX-Slot abzuarbeiten (Präambelsuche, FFT, LDPC).
3. **TX-Slot (Knoten B) / RX-Slot (Knoten A):** 100 – 120 ms. Rollentausch.
4. **Verarbeitungs-Slot (Knoten A):** 120 – 200 ms.

Umsetzung in Python: Du musst Threading nutzen. Ein Thread betreibt den SDR und liest/schreibt I/Q-Daten strikt nach Timer. Ein zweiter Thread bearbeitet die Puffer. Das Stoppen des Empfängers (z.B. Zerstören des `libiio.Buffer`) ist wichtig, damit das Betriebssystem den DMA-Controller des FPGAs anweist, keine weiteren Daten in den PC-Speicher zu pushen, da sonst während des rechenintensiven Verarbeitungs-Slots Überläufe auftreten.

P19 – Gemini 2.5 Pro (16.05.2026)

Beschreibung: Vorschläge zur Erfassung der PPS-Flanken, zur seriellen Übertragung und zur Überprüfung der zeitlichen Abstände. Diente als Grundlage für die spätere GPS-Integration.

Exakter Prompt:

```
Ich habe ein GPS-Modul an einen ESP32 gehängt. Wie lese ich den 1-PPS Pin per Interrupt aus und sende ihn per UART an meinen Laptop, um beide SDR-Knoten auf die Mikrosekunde zu synchronisieren?
```

Antwort der KI:

Die Synchronisation zweier getrennter SDR-Systeme über GPS ist zwingend, um ein TDMA-Schema oder TDoA (Time Difference of Arrival) aufzubauen. Das 1-PPS (Pulse Per Second) Signal eines GPS-Empfängers liefert eine globale Zeitreferenz mit einer Genauigkeit von < 50 ns.

Vorgehen mit einem ESP32 als Zeit-Stempel-Einheit:

1. **Hardware-Interrupts:** Der ESP32 verbindet seinen GPIO-Pin mit dem PPS-Ausgang. Dieser Pin wird als Interrupt (`RISING` Edge) konfiguriert.
2. **Mikrosekunden-Timer:** Sobald der Interrupt auslöst, ruft die ISR (Interrupt Service Routine) sofort den Hardware-Timer ab: `uint32_t timestamp = micros();`. Die ISR darf nichts anderes tun, um Jitter zu vermeiden.
3. **UART-Übertragung:** Im Haupt-Loop prüft der ESP32, ob ein neuer Stempel vorliegt, und sendet diesen per UART (`Serial.println()`) an den Host-PC (z.B. Raspberry Pi oder Laptop, an dem das SDR hängt).

Software-Synchronisation auf PC-Seite: Der PC liest kontinuierlich I/Q-Samples. Gleichzeitig wartet ein Thread auf die UART-Nachricht des ESP32.

- Sobald die Nachricht eintrifft, notiert der PC den aktuellen Sample-Index N_{sync} des I/Q-Datenstroms.
- Da UART langsam ist, gibt es eine Latenz. Diese Latenz Δt_{uart} muss man messen oder schätzen.
- Der "echte" PPS-Moment im I/Q-Datenstrom lag also $\Delta t_{uart} \cdot f_s$ Samples in der Vergangenheit.

Beide Knoten starten ihren TDMA-Zähler exakt ab diesem kalibrierten Sample-Index neu. So wissen beide Systeme bis auf wenige Samples genau, wann eine neue Sekunde begonnen hat.

P20 – Gemini 2.5 Pro (29.06.2026)

Beschreibung: Vorgehen zur isolierten Messung von Pufferzustand, Laufzeit und Verarbeitungsdauer. Die Hinweise wurden für eigene Python-Testskripte und die Latenzanalyse verwendet.

Exakter Prompt:

Ich verliere bei längeren Übertragungen gelegentlich Datenpakete. Wie kann ich messtechnisch sauber nachweisen, ob dies am Funkkanal liegt oder ob libiio überläuft und Buffer Overruns verursacht?

Antwort der KI:

Verpasste Signalpakete sind in SDR-Systemen schwer zu debuggen. Die Ursachen können RF-Interferenzen (schlechter SNR), Timing-Jitter oder reine Software-Latenzen (Buffer Overruns) sein. Um gezielt Buffer Overruns nachzuweisen, muss man den Software-Pfad isolieren.

Vorgehen zur isolierten Analyse:

1. **Puffer-Füllstand (Queue-Monitoring):** Wenn du einen Thread-basierten Ansatz nutzt (Thread 1: SDR lesen → Queue → Thread 2: DSP), miss kontinuierlich die Grösse der Queue (`queue.qsize()`). Wächst die Queue stetig an, ist deine Verarbeitungsdauer im Durchschnitt langsamer als die Abtastrate. Es wird zwangsläufig zum Crash kommen.
2. **Isolierte Laufzeitmessung (Profiler):** Baue in deinen Code Zeitstempel um die rechenintensiven Blöcke:

```
t0 = time.perf_counter()
do_fft_and_correlate(buffer)
t1 = time.perf_counter()
run_ldpc_decoder(payload)
t2 = time.perf_counter()
print(f"DSP: {(t1-t0)*1000}ms, LDPC: {(t2-t1)*1000}ms")
```

Vergleiche diese Summe zwingend mit der Block-Dauer ($T_{block} = N_{samples}/f_s$).

3. **Erkennen des Hardware-Overruns:** Ein Puffer-Verlust in der Kernel-Treiberschicht wird dir im Programmcode oft nicht als Exception geworfen. Du erkennst ihn an *Phasensprüngen* oder einem Riss in einer Sinuswelle, wenn du die Samples plottest. Manche Treiber (z.B. UHD) geben ein konsolenbasiertes "O" für Overrun aus. Bei libiio musst du den Rückgabewert von `buffer.refill()` prüfen. Ist er kleiner als die angeforderte Bytezahl, gab es einen Drop.
-

P21 – Gemini 2.5 Pro (02.07.2026)

Beschreibung: Vorschläge für strukturierte Parametervergleiche und reproduzierbare Testreihen. Unterstützung bei der Planung der eigenen Messreihen.

Exakter Prompt:

Ich möchte die BER in Abhängigkeit von Bandbreite und SF messen. Wie baue ich dafür am besten einen reproduzierbaren Test-Loop (z.B. mit I/Q-Dumps) auf, um Schwankungen durch externe Interferenzen im Labor auszuschliessen?

Antwort der KI:

Um die Auswirkungen verschiedener PHY-Parameter (Bandbreite, Abtastrate, Spreading Factor, Coding Rate) auf die Bit Error Rate (BER) systematisch zu evaluieren, darf man niemals in der Live-Funk-Umgebung messen. Funkkanäle fluktuieren (Fading, Interferenzen), wodurch Messreihen nicht reproduzierbar sind.

Strukturierter Parametervergleich (Reproduzierbare Testreihen):

1. **I/Q-Dumps aufzeichnen:** Nimm ein Referenz-Szenario auf. Sende z.B. 100 Pakete und speichere die rohen Basisband I/Q-Daten des Empfängers direkt in eine `.bin`-Datei (als float32 oder int16 Array), ohne sie zu decodieren.
2. **Synthetisches Rauschen (AWGN) hinzufügen:** Um den SNR kontrolliert zu variieren, schreibst du ein Skript, das die saubere `.bin`-Datei einliest und exakt berechnetes Gaussian Noise hinzufügt:

$$I_{noisy} = I + \text{random.gauss}(0, \sigma)$$

3. **One-Factor-At-A-Time (OFAT):** Schreibe ein Test-Skript, das die Offline-Pipeline aufruft. Ändere immer nur eine Variable (z.B. den Schwellwert für die Präambel-Erkennung) und lasse die 100 synthetischen Pakete durchlaufen. Speichere die *Packet Error Rate* (PER).
 4. **Systematischer Vergleich (Grid Search):** Erstelle eine Matrix mit den Variationen. Z.B. vergleiche Bandbreite (125 kHz vs 250 kHz) bei konstantem Spreading Factor in einer simulierten Umgebung. Da die I/Q-Dateien immer exakt gleich sind, ist jede Änderung der BER ausschliesslich auf deine Parameteränderung zurückzuführen.
-

P22 – Gemini 2.5 Pro (27.07.2026)

Beschreibung: Konzept für die Suche nach Korrelationskandidaten und anschliessende Verifikation der Payload. Grundlage für die eigene Synchronisationslogik und deren automatisierten Test.

Exakter Prompt:

Um CPU zu sparen, will ich eine zweistufige Präambelsuche einbauen: Zuerst ganz grob nach Energie suchen, dann genau. Wie strukturiere ich diese Brute-Force-Suche softwaretechnisch?

Antwort der KI:

Eine zweistufige Brute-Force-Suche in einem I/Q-Datenstrom ist ein effizienter Weg, um Signale (wie LoRa oder QPSK-Pakete) extrem robust, aber mit reduzierter durchschnittlicher Rechenlast zu finden. Das Ziel ist es, False-Positives (Phantom-Signale) frühzeitig herauszufiltern, bevor man teure Algorithmen wie LDPC bemüht.

Konzept der zweistufigen Suche:

Stufe 1: Grobsuche (Fast Detection) Hier wird eine computationally "billige" Metrik verwendet. Bei LoRa nutzt man oft die Energie-Detektion nach der De-Chirp-Multiplikation.

- Man nimmt einen Puffer von N Samples, führt die De-Chirp-Operation durch und wendet eine kleine FFT an.
- Man sucht nach dem maximalen Peak: $P_{max} = \max(|X(k)|^2)$.
- Liegt P_{max} über einem definierten *Threshold* (der recht tief angesetzt wird, um keine echten Pakete zu übersehen), wird der Puffer als "Kandidat" markiert.

Stufe 2: Feinsuche & Verifikation (Payload Decoding) Nur wenn Stufe 1 triggert, wird die rechenintensive Pipeline aktiviert.

- **Zeitliche Synchronisation:** Man prüft, ob in den darauffolgenden Zeitfenstern das spezifizierte Sync-Word-Muster (z.B. invertierte Down-Chirps) exakt auftritt.
- **Demodulation:** Ist das Sync-Word gefunden, demoduliert man die Symbole zu LLRs (Log-Likelihood Ratios).
- **Verifikation durch FEC:** Das finale Kriterium ist der Vorwärtsfehlerkorrektur-Decoder (z.B. LDPC oder Viterbi). Wenn der LDPC-Decoder nach X Iterationen ein gültiges Syndrom ($\mathbf{H} \cdot \mathbf{v} = 0$) meldet, weiss das System mit nahezu 100% Sicherheit, dass es sich um eine gültige Payload und nicht um Rauschen handelt.

P23 – Gemini 2.5 Pro (28.07.2026)

Beschreibung: Erklärung des Prinzips der DQPSK und Vorschläge zur Differenzbildung zwischen aufeinanderfolgenden Symbolen. Grundlage für die eigene Implementierung und anschliessende Tests.

Exakter Prompt:

Was bringt mir eigentlich differentielle QPSK (DQPSK) im Vergleich zur normalen absoluten QPSK-Phasenübertragung? Erkläre mir das Konzept und wie ich in Python die Phasendifferenz auswerte.

Antwort der KI:

Die absolute QPSK-Übertragung (Quadrature Phase Shift Keying) codiert die Bit-Informationen (00, 01, 10, 11) direkt in die vier Phasenwinkel (45° , 135° , 225° , 315°) des Trägersignals. Das massive Problem dabei: Jeder Carrier Frequency Offset (CFO) oder jede Veränderung der Ausbreitungszeit führt zu einer unbekannten Phasenrotation des gesamten Diagramms. Der Empfänger weiss nicht, wo 0° ist (Phasen-Ambiguität).

Prinzip der Differentiellen QPSK (DQPSK): Anstatt den absoluten Winkel eines Symbols zu bewerten, codiert man die Information in die **Phasendifferenz** zwischen dem aktuellen Symbol n und dem vorherigen Symbol $n - 1$.

$$\Delta\phi_n = (\phi_n - \phi_{n-1}) \pmod{360^\circ}$$

Warum das Phasenrotationen eliminiert: Angenommen, der Kanal fügt einen statischen oder langsam veränderlichen Phasenoffset ϕ_{error} hinzu. Der Empfänger misst:

- $\phi_n^{rx} = \phi_n^{tx} + \phi_{error}$
- $\phi_{n-1}^{rx} = \phi_{n-1}^{tx} + \phi_{error}$

Bildet der Empfänger nun die Differenz:

$$\Delta\phi_n^{rx} = (\phi_n^{tx} + \phi_{error}) - (\phi_{n-1}^{tx} + \phi_{error}) = \phi_n^{tx} - \phi_{n-1}^{tx} = \Delta\phi_n^{tx}$$

Der Störterm ϕ_{error} fällt mathematisch exakt heraus!

Implementierungsvorschlag (Python): Anstatt komplexe Winkel zu subtrahieren, multipliziert man das aktuelle komplexe Sample s_n mit dem konjugiert komplexen des vorherigen s_{n-1}^* :

```
import numpy as np
# arr ist ein Array mit komplexen Symbolen
delta_symbols = arr[1:] * np.conjugate(arr[:-1])
# Der Winkel von delta_symbols enthält direkt die Differenz
diff_phases = np.angle(delta_symbols)
```

P24 – Gemini 2.5 Pro (29.07.2026)

Beschreibung: Hinweise zur Verschaltung, Signalverarbeitung und Überprüfung des PPS-Signals. Unterstützung bei der praktischen GPS-Integration.

Exakter Prompt:

Ich nutze ein ATGM336H GPS-Modul am ESP32. Das 1-PPS-Signal ist extrem kurz (High-Pegel). Wie konfiguriere ich die C++ Interrupt-Routine so, dass das Timing absolut stabil bleibt?

Antwort der KI:

Das ATGM336H ist ein weit verbreitetes BDS/GPS-Modul, das ein sehr präzises 1-PPS (Pulse Per Second) Signal ausgibt. Dieses Signal bleibt für einen Bruchteil einer Sekunde HIGH und markiert exakt den Start einer neuen UTC-Sekunde.

Um dieses Signal mit einem ESP32 zuverlässig als Hardware-Timing einzulesen, musst du Polling im Haupt-Loop vermeiden und stattdessen Hardware-Interrupts nutzen.

Verschaltung und Signalverarbeitung:

1. **Verkabelung:** Verbinde den PPS-Pin des ATGM336H mit einem GPIO-Pin des ESP32, der Interrupts unterstützt (z.B. GPIO 25). Die Spannungspegel (3.3V) sind kompatibel.
2. **Interrupt Setup (Arduino/C++):** Konfiguriere den Pin als Input und hänge eine ISR (Interrupt Service Routine) an, die auf die steigende Flanke ('RISING') reagiert.

```
volatile uint32_t pps_timestamp = 0;
volatile bool pps_flag = false;

// IRAM_ATTR zwingt die ISR in den schnellen RAM
void IRAM_ATTR pps_isr() {
    pps_timestamp = micros(); // Exakten Timer-Wert holen
    pps_flag = true;
}

void setup() {
    pinMode(25, INPUT_PULLDOWN);
    attachInterrupt(digitalPinToInterrupt(25), pps_isr, RISING);
}
```

3. **Verarbeitung im Loop:** Die ISR darf **niemals** zeitaufwändige Aufgaben wie 'Serial.print' enthalten. Sie setzt nur das Flag. Im Hauptprogramm reagierst du darauf:

```
void loop() {
    if (pps_flag) {
        pps_flag = false;
        // Jetzt den Zeitstempel z.B. an den PC senden
        Serial.printf("PPS: %u\n", pps_timestamp);
    }
}
```

Zur Überprüfung schickst du die Zeitstempel an den PC. Die Differenz zwischen zwei pps_timestamp-Werten muss exakt 1.000.000 Mikrosekunden betragen (plus/minus den Jitter des ESP32 Quarzes).

P25 – Gemini 2.5 Pro (30.07.2026)

Beschreibung: Python-Struktur zum Einlesen von I/Q-Dumps, Iterieren über Kandidaten und automatischen Prüfen der Payload. Als Grundlage für das eigene Testskript angepasst.

Exakter Prompt:

Schreibe mir ein Python-Template, mit dem ich rohe .bin-I/Q-Dumps einlese, in Chunks zerschneide und damit einen automatisierten Regression-Test gegen meine Demodulator-Funktion laufen lassen kann.

Antwort der KI:

Ein automatisierter Offline-Test für I/Q-Daten ist der Goldstandard bei der Entwicklung von Demodulations-Algorithmen. Er ermöglicht es, Code-Änderungen sofort gegen bekannte, reale Funk-Szenarien zu validieren (Regression-Testing).

Struktur des Python-Testskripts:

1. **Einlesen des Dumps:** Lade die aufgezeichneten I/Q-Rohdaten (oft als 32-Bit Float oder 16-Bit Int gespeichert).

```
import numpy as np

# Angenommen Datei ist interleaved int16: I,Q,I,Q...
raw = np.fromfile('dump.bin', dtype=np.int16)
iq_data = raw[0::2] + 1j * raw[1::2]
```

2. **Streaming Simulation (Chunking):** Dein Echtzeit-System liest Daten in Blöcken. Simuliere das, indem du das Array in Chunks teilst:

```
chunk_size = 4096
for i in range(0, len(iq_data), chunk_size):
    chunk = iq_data[i : i+chunk_size]

    # Sende Chunk an deine Pipeline
    # candidates = preamble_detector.process(chunk)
```

3. **Kandidaten-Iteration & Payload-Verifikation:** Die Demodulationspipeline liefert oft mehrere Kandidaten (z.B. Peaks, die den Schwellwert übersteigen). Du iterierst über diese Kandidaten, startest die Demodulation und prüfst die decodierte Payload gegen einen bekannten Wert:

```
expected_payload = b"Hello SDR!"
success_count = 0

for candidate_offset in candidates:
    payload, is_valid = demodulator.decode(iq_data, candidate_offset)
    if is_valid and payload == expected_payload:
        success_count += 1
    print(f"Paket gefunden bei Offset {candidate_offset}")
```

```
if success_count == 0:
    print("Regression: Paket konnte nicht gefunden werden!")
```

P26 – Gemini 2.5 Pro (02.08.2026)

Beschreibung: Hinweise zur Maskierung von Sonderzeichen, zur `listings`-Konfiguration und zur robusten Darstellung von Python-Code. Die Änderungen wurden selbst übernommen und kontrolliert.

Exakter Prompt:

Ich erhalte in LaTeX die Fehlermeldung 'Missing \$ inserted', wenn ich im Fliesstext oder im Codeblock die Variable `qpsk_wert` verwende. Wie kann ich diesen Fehler beheben, ohne dass die Formatierung des Python-Codes beeinträchtigt wird?

Antwort der KI:

Das Problem bei der Einbindung von Programmcode oder Variablennamen (wie `qpsk_wert`) in LaTeX-Dokumenten ist, dass LaTeX den Unterstrich (`_`) standardmässig als Beginn eines Index (Subscript) im mathematischen Modus interpretiert.

Behebung für Inline-Text: Wenn du Variablennamen direkt im Fliesstext erwähnen möchtest, musst du den Unterstrich mit einem Backslash maskieren und solltest eine Monospace-Schrift verwenden, um ihn als Code zu kennzeichnen:

Hier wird die Variable `\texttt{qpsk\_wert}` berechnet.

(Falsch wäre: `qpsk_wert`, da dies einen Fehler wirft: "Missing \$ inserted").

Behebung für Code-Listings (Blöcke): Für grössere Code-Blöcke ist manuelles Maskieren extrem fehleranfällig und unleserlich. Verwende zwingend das `listings` (oder `minted`) Paket. Innerhalb der `lstlisting`-Umgebung schaltet LaTeX die Sonderzeichen-Auswertung ab.

```
% In der Präambel:
\usepackage{listings}
\lstset{
    language=Python,
    basicstyle=\ttfamily\small,
    breaklines=true,
    showstringspaces=false
}

% Im Dokument:
\begin{lstlisting}
def calculate_phase(qpsk_wert):
    # Hier muss kein Unterstrich maskiert werden!
    return np.angle(qpsk_wert)
```

\end{lstlisting}

Durch das `basicstyle=\ttfamily` wird sichergestellt, dass der Code in einer sauberen Festbreitenschrift gedruckt wird, ohne dass das restliche Layout verschlechtert wird.

P27 – Gemini 2.5 Pro (04.08.2026)

Beschreibung: Ansatz für kurze Empfangsbursts innerhalb eines TDMA-Slots und anschliessende Verarbeitung von FFT und LDPC ausserhalb der Aufnahmezeit. Grundlage für die eigene Umsetzung.

Exakter Prompt:

Erklär mir das Konzept eines TDMA-Burst-Modus genauer: Wenn ich exakt bei $t=0$ vom GPS einen kurzen SDR-Burst aufnehme, wie stelle ich sicher, dass der Puffer in der restlichen Slot-Zeit ruht?

Antwort der KI:

Ein TDMA (Time Division Multiple Access) Burst-Modus, der über GPS synchronisiert wird, ist die Lösung, wenn eine CPU zu schwach ist, um einen kontinuierlichen SDR-Stream lückenlos in Echtzeit zu demodulieren.

Das Ziel ist ein *Duty Cycle*, bei dem der PC nur einen Bruchteil des Zeit-Slots Daten aufnimmt und den gesamten restlichen Slot für die rechenintensive FFT und LDPC-Decodierung nutzt.

Konzept des Software-Ablaufs: Ein TDMA-Slot dauere genau 100 ms. Ein GPS-PPS-Signal liefert den Start-Trigger ($t = 0$).

- 1. Aufnahmezeit (Burst) - $t = 0$ bis 5 ms:** Bei Trigger-Eingang aktiviert der Host-PC den SDR-RX-Stream (`libiio.Buffer`). Wir lesen exakt die Anzahl an Samples, die in ein 5-ms-Fenster passen ($N = 5\text{ms} \cdot f_s$). In dieser Zeit läuft die CPU-Verarbeitung auf minimaler Last, um sicherzustellen, dass das Array ohne DMA-Blockaden in den RAM geschrieben wird.
- 2. RX-Shutdown / Discard - $t = 5$ ms:** Unmittelbar nachdem die N Samples gelesen wurden, wird der Puffer-Stream **gestoppt** (`buffer.cancel()` oder `rx_channel.enabled = False`). Dies ist der entscheidende Schritt: Der USB/Netzwerk-Bus wird stillgelegt. Das Betriebssystem (Linux) und der SDR-Treiber erhalten keine Interrupts mehr vom FPGA. Es drohen keine Overruns mehr, egal wie langsam die CPU ab jetzt ist.
- 3. Rechenintensive Decodierung - $t = 5$ ms bis 95 ms:** Die CPU hat nun üppige 90 ms Zeit, um das kleine, 5 ms lange I/Q-Array zu verarbeiten. Hier laufen nun in aller Ruhe die Kreuzkorrelation, De-Chirping-Multiplikationen, grosse FFT-Arrays und der iterative LDPC-Decoder. Da keine neuen USB-Pakete die CPU unterbrechen, ist die Laufzeit sehr stabil.
- 4. Idle & Warten auf Sync - $t = 95$ ms bis 100 ms:** Die Pipeline ist abgeschlossen. Der Thread geht in den `sleep()`-Modus oder wartet am Semaphore auf das nächste Hardware-Interrupt-Ereignis des GPS-Moduls (oder eines Hardware-Timers), das den Start des nächsten 100-ms-Frames signalisiert.

Umsetzung in Python:

```
import time

def tdma_worker():
    while True:
        # Blockiert, bis GPS-Interrupt oder Timer das Event setzt
        wait_for_gps_sync()

        # 1. Burst-Aufnahme
        # Buffer z.B. so konfiguriert, dass er exakt 5ms fasst
        buf.refill()
        raw_iq = buf.read()

        # 2. SDR-Puffer invalidieren/ignorieren für den Rest des Slots

        # 3. Intensive Verarbeitung (ohne Stress)
        iq_samples = convert_to_complex(raw_iq)
        payload = heavy_dsp_pipeline(iq_samples)

        if payload_is_valid(payload):
            print("Payload erfolgreich extrahiert.")

        # Loop beginnt von vorn, wartet wieder im Idle-Zustand
```

Dieser Ansatz eliminiert das klassische "Live-Streaming"-Problem komplett und opfert dafür lediglich Empfangszeit (Duty Cycle von 5%), was bei synchronisierten Protokollen, bei denen das Sende-Timing bekannt ist, völlig akzeptabel ist.
